

Project #2 guidelines for Python

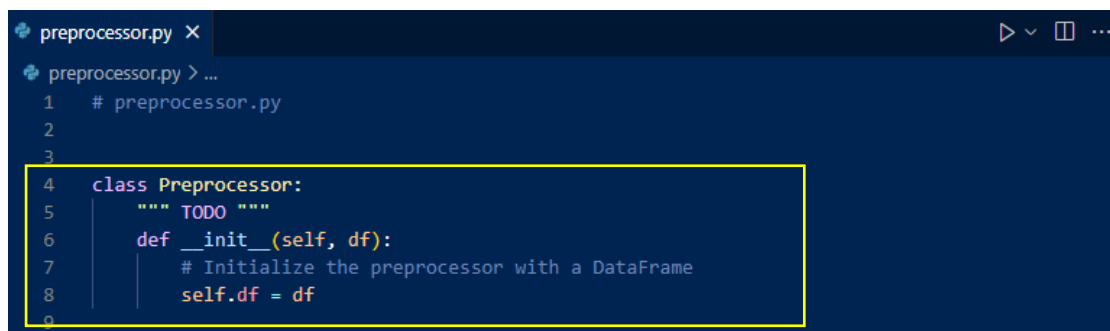
If you decide to write your project using python, I'll tell you where to implement the code in project #2 framework.

Implementation

1. Use your custom data preprocessing methods

- a. Complete ***Preprocessor class*** in *preprocessor.py*

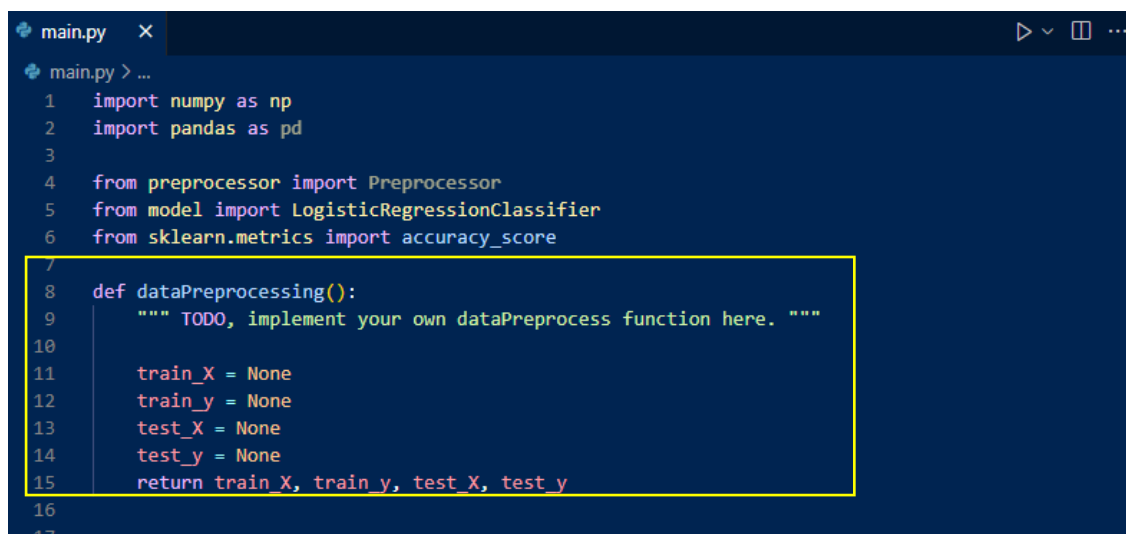
Use your custom data preprocessing functions in project #1 as the methods in Preprocessor class.



```
preprocessor.py X
preprocessor.py > ...
1 # preprocessor.py
2
3
4 class Preprocessor:
5     """ TODO """
6     def __init__(self, df):
7         # Initialize the preprocessor with a DataFrame
8         self.df = df
9
```

- b. Complete ***DataPreprocessing function*** in *main.py*

Use the custom methods in project #1 you implemented to complete dataPreprocessing function.



```
main.py X
main.py > ...
1 import numpy as np
2 import pandas as pd
3
4 from preprocessor import Preprocessor
5 from model import LogisticRegressionClassifier
6 from sklearn.metrics import accuracy_score
7
8 def dataPreprocessing():
9     """ TODO, implement your own dataPreprocess function here. """
10
11     train_X = None
12     train_y = None
13     test_X = None
14     test_y = None
15     return train_X, train_y, test_X, test_y
16
17
```

Note: You can either use your data preprocessing methods in project #1 or implement a new one for project #2.

2. Complete MLP in *model.py*

a. Implement necessary methods in *MLPClassifier*

```
class MLPClassifier(Classifier):
    def __init__(self, layers, activate_function, optimizer, learning_rate, n_epoch = 1000):
        """ TODO, Initialize your own MLP class """

        self.layers = layers
        self.activate_function = activate_function
        self.optimizer = optimizer
        self.learning_rate = learning_rate
        self.n_epoch = n_epoch

    def forwardPass(self, X):
        # TODO
        pass

    def backwardPass(self, y):
        # TODO
        pass

    def update(self):
        """ The update function to update parameters """
        # TODO
        pass

    def fit(self, X, y):
        # TODO
        pass

    def predict(self, X):
        y_hat = self.predict_proba(X)
        return np.array([1 if i > 0.5 else 0 for i in y_hat])

    def predict_proba(self, X):
        return self.forwardPass(X)
```

b. Implement *activation class* and *optimizer class*

- Write all your activation functions (sigmoid, relu, ...) as methods in activation class
- Write all your optimizers as methods in optimizer class (SGD, Adagrad, ...)

```
model.py x
model.py > ...
1  import numpy as np
2  from abc import ABC, abstractmethod
3
4  # ===== Activation function ===== #
5  class activation():
6      def __init__(self):
7          # TODO
8          pass
9
10
11 # ===== Optimizer function ===== #
12 class optimizer():
13     def __init__(self):
14         # TODO
15         pass
```

Note: you can use sigmoid and SGD I implemented in project #1

Some examples of well-known optimizers:

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ , momentum parameter α .

Require: Initial parameter θ , initial velocity v .

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \mathbf{g}$

 Apply update: $\theta \leftarrow \theta + \mathbf{v}$

end while

(Ian, Goodfellow, et al. *Deep learning, p298)

Algorithm 8.4 The AdaGrad algorithm

Require: Global learning rate ϵ

Require: Initial parameter θ

Require: Small constant δ , perhaps 10^{-7} , for numerical stability

Initialize gradient accumulation variable $\mathbf{r} = \mathbf{0}$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Accumulate squared gradient: $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g}$

 Compute update: $\Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g}$. (Division and square root applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta\theta$

end while

(Ian, Goodfellow, et al. *Deep learning, p308)

Algorithm 8.5 The RMSProp algorithm

Require: Global learning rate ϵ , decay rate ρ .

Require: Initial parameter θ

Require: Small constant δ , usually 10^{-6} , used to stabilize division by small numbers.

Initialize accumulation variables $\mathbf{r} = \mathbf{0}$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Accumulate squared gradient: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$

 Compute parameter update: $\Delta\theta = -\frac{\epsilon}{\sqrt{\delta + \mathbf{r}}} \odot \mathbf{g}$. ($\frac{1}{\sqrt{\delta + \mathbf{r}}}$ applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta\theta$

end while

(Ian, Goodfellow, et al. *Deep learning, p309)

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization. (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

(Ilan, Goodfellow, et al. *Deep learning, p311)